Viridis Research — certification record

Run `runs/2026-10-27_P1` 2026-10-27 project P1, task P1-T001 engine `6441f38ed19c`
GitHub run: `https://github.com/jdhart81/viridis-engine/actions/runs/40605294004`

A Lean Proof of the Bit-Quad Euler Formula for 8-Connected Rasters: the First Rung of a Raster Bridge for Trees Cannot Cut
Justin Hart, Viridis LLC

Labels awarded by the verifier

Claim	Label	Claim text (as submitted)
C1	**PROVEN**	formal [P1.BitQuad]: For every finite set B of pixels in Z x Z, 4 * euler B = Q1 B - Q3 B - 2 * QD B (in Z), where euler B = V - E + F of the cubical complex of B and Q1, Q3, QD count 2x2 windows centred on vertices of K(B) with one pixel, three pixels, or a diagonal pair.

Sealed statement P1.BitQuad

File `lean/ViridisStatements/P1/BitQuad.lean`, sha256 `cd3cdeba48f67f9545fb6945c87037893aa98b018922d3d0b7220cc389c98e23`, sealed by decision D-001. Axioms used by the proof: Classical.choice, Quot.sound, propext. Candidate sha256 `82dc3598b4fab8a6c72fc6fe4021eef1ede516530d2cc28b077a4acf0003c056`. Listing shown with symbol substitution; the exact bytes are in the repository.

```
import Mathlib

/-!
# P1 · L1: the bit-quad formula for an 8-connected raster

A raster is a finite set `B` of pixels in `ℤ × ℤ`. Pixel `(i, j)` stands for the
closed unit square `[i, i+1] × [j, j+1]`. `K B` is the union of the closed
squares of `B`. It is a cubical complex, and its vertices, edges and faces are
counted below. Two pixels that share only a corner meet in `K B`, which is the
8-connected reading of the foreground.

`Statement` says that for every finite raster,
`4 · χ(K B) = Q₁ − Q₃ − 2·Q_D`. Here `Q₁`, `Q₃` and `Q_D` count the 2×2 windows
that hold exactly one pixel of `B`, exactly three pixels of `B`, and exactly two
diagonally opposite pixels of `B` (Gray 1971).

This is the first rung (L1) of the P1 lemma ladder. See
`projects/P1/context/STATEMENT_PROPOSAL.md`.
-/

namespace ViridisStatements.P1.BitQuad

/-- A pixel, named by its lower-left lattice corner. -/
abbrev Pixel := ℤ × ℤ

/-- The four lattice corners of pixel `p`. -/
def corners (p : Pixel) : Finset (ℤ × ℤ) :=
  {(p.1, p.2), (p.1 + 1, p.2), (p.1, p.2 + 1), (p.1 + 1, p.2 + 1)}

/-- Vertices of `K B`: every corner of every pixel of `B`. -/
def vertices (B : Finset Pixel) : Finset (ℤ × ℤ) :=
  B.biUnion corners

/-- Horizontal unit edges `[(i, j), (i + 1, j)]` of `K B`, named by their left
end. These are the bottom edge `(i, j)` and the top edge `(i, j + 1)` of each
pixel `(i, j)` of `B`. -/
def hEdges (B : Finset Pixel) : Finset (ℤ × ℤ) :=
  B.biUnion fun p => {(p.1, p.2), (p.1, p.2 + 1)}

/-- Vertical unit edges `[(i, j), (i, j + 1)]` of `K B`, named by their lower
end. These are the left edge `(i, j)` and the right edge `(i + 1, j)` of each
pixel `(i, j)` of `B`. -/
def vEdges (B : Finset Pixel) : Finset (ℤ × ℤ) :=
  B.biUnion fun p => {(p.1, p.2), (p.1 + 1, p.2)}

/-- The Euler characteristic `V − E + F` of `K B`. -/
def euler (B : Finset Pixel) : ℤ :=
  ((vertices B).card : ℤ) - (((hEdges B).card : ℤ) + (vEdges B).card) + B.card

/-- The 2×2 window centred on the lattice point `v`, as four membership bits in
this order: lower-left, lower-right, upper-left, upper-right. -/
```

```
def bits (B : Finset Pixel) (v : ℤ × ℤ) : Bool × Bool × Bool × Bool :=
  (decide ((v.1 - 1, v.2 - 1) ∈ B), decide ((v.1, v.2 - 1) ∈ B),
   decide ((v.1 - 1, v.2) ∈ B), decide ((v.1, v.2) ∈ B))

/-- The number of pixels of `B` in the window centred on `v`. -/
def windowCount (B : Finset Pixel) (v : ℤ × ℤ) : ℕ :=
  match bits B v with
  | (a, b, c, d) => a.toNat + b.toNat + c.toNat + d.toNat

/-- True when the window centred on `v` holds exactly two diagonally opposite
pixels of `B`. -/
def isDiagonal (B : Finset Pixel) (v : ℤ × ℤ) : Bool :=
  match bits B v with
  | (a, b, c, d) => (a && d && !b && !c) || (b && c && !a && !d)

/-- Windows holding exactly one pixel of `B`. Every window that meets `B` is
centred on a vertex of `K B`, so counting over `vertices B` counts each such
window exactly once. -/
def Q1 (B : Finset Pixel) : ℕ :=
  ((vertices B).filter fun v => windowCount B v = 1).card

/-- Windows holding exactly three pixels of `B`. -/
def Q3 (B : Finset Pixel) : ℕ :=
  ((vertices B).filter fun v => windowCount B v = 3).card

/-- Windows holding exactly two diagonally opposite pixels of `B`. -/
def QD (B : Finset Pixel) : ℕ :=
  ((vertices B).filter fun v => isDiagonal B v = true).card

/-- **Sealed target `P1.BitQuad`.** For every finite raster `B`,
`4 · χ(K B) = Q₁ − Q₃ − 2·Q_D`. -/
def Statement : Prop :=
  ∀ B : Finset Pixel, 4 * euler B = (Q1 B : ℤ) - Q3 B - 2 * QD B

/-! ## Sanity checks

These checks are proved by computation inside the sealed module, so the
definitions above mean what their comments say. -/

/-- A single pixel: `V = 4`, `E = 4`, `F = 1`, so `χ = 1`, and its four windows
each hold one pixel. -/
example : euler {(0, 0)} = 1 ∧ Q1 {(0, 0)} = 4 ∧ Q3 {(0, 0)} = 0 ∧ QD {(0, 0)} = 0 := by
  decide

/-- Two pixels that touch only at a corner form one 8-connected piece: `χ = 1`,
and the shared corner is a diagonal window. -/
example : euler {(0, 0), (1, 1)} = 1 ∧ QD {(0, 0), (1, 1)} = 1 ∧ Q1 {(0, 0), (1, 1)} = 6 := by
  decide

/-- A 2×2 solid block is contractible: `χ = 1`. -/
example : euler {(0, 0), (1, 0), (0, 1), (1, 1)} = 1 := by
  decide

/-- The 3×3 ring (eight pixels around one hole) has `χ = 0`. -/
example : euler {(0, 0), (1, 0), (2, 0), (0, 1), (2, 1), (0, 2), (1, 2), (2, 2)} = 0 := by
  decide

end ViridisStatements.P1.BitQuad
```

Other checks

- Lean toolchain leanprover/lean4:v4.28.0, Mathlib 8f9d9cff6bd7; kernel replay by `leanchecker`.
- References: 5 entries, 5 DOIs, 5 resolved, 0 unresolved, 0 unchecked.
- Agent paper: compiled.
- Statement proposals: 0.

How to read this record

ASSUMED: not verified (the default). **PROVEN**: a Lean 4 proof of a statement sealed before the attempt, compiled with the pinned toolchain, matched definitionally to the sealed statement, using only the axioms propext, Classical.choice and Quot.sound, and replayed through the kernel. **TESTED/REPLICATED**: reproduced by the verifier (not yet enabled). **ENDORSED/ADOPTED**: granted by people. Novelty is never a label. **TRIVIAL** marks a sealed statement that one of the engine's probe tactics already closes.

This paper was drafted by an AI research agent (Claude, by Anthropic) running the Viridis research engine; formal proof search may use Aristotle (Harmonic). Justin Hart directs the research program. The labels on this page were assigned by an automated verifier, not by the agent, and only they carry certification. Claims in the paper that are not listed here as PROVEN are ASSUMED.

Reproduce everything from `REPRODUCE.md` in the run folder; the run's files are listed with hashes in `MANIFEST.sha256`.

A Lean Proof of the Bit-Quad Euler Formula for 8-Connected Rasters: the First Rung of a Raster Bridge for *Trees Cannot Cut*

Justin Hart
Viridis LLC

October 2026

Abstract

Trees Cannot Cut argues that fuel breaks and habitat corridors split a landscape into compartments only through their cycles. Its machine-checked core carries the topological step (compartments equal circuit rank plus one) as an explicit hypothesis. Project P1 aims to replace that hypothesis, on finite rasters, with a proved identity: enclosed non-barrier 4-components equal $C_8(B) - \chi(KB)$, where KB is the cubical complex of the barrier raster B. This paper does the first rung of that ladder. We give a Lean 4/Mathlib proof of Gray's bit-quad formula in its 8-connected form: for every finite raster $B \subset \mathbb{Z}^2$, $4\chi(KB) = Q_1 - Q_3 - 2Q_D$, where Q_1, Q_3 and Q_D count the 2×2 windows holding exactly one pixel, exactly three pixels, or exactly two diagonally opposite pixels of B. The proof double-counts over the vertices of KB. Shift bijections show that $4F$, $2E$ and V are sums of local window quantities. A 16-case analysis then settles the pointwise identity $4 - 2\,\mathrm{arms}(v) + n(v) = [n = 1] - [n = 3] - 2[\mathrm{diag}]$, which holds on every non-empty window (it fails on the empty window, which never sits at a vertex). The development is about 160 lines, uses no custom axioms, and type-checks against a sealed statement module. We explain how L1 feeds the single-pixel insertion lemma (L2) and the bridge (L3).

1 Introduction

Hart's *Trees Cannot Cut* [3] makes a structural claim about managed forest landscapes. A corridor or fuel-break network separates the fuel matrix into compartments only through its cycles, so an acyclic network (a "tree") cannot cut. The manuscript's Lean core proves the graph-theoretic consequences of this claim. The topological input is assumed as a hypothesis, `FaceCount`: the number of compartments is circuit rank plus one. It is justified informally by Alexander duality, which Mathlib does not have.

Fire and fuel data usually come as rasters, and on rasters the topology is finite and combinatorial. Project P1 therefore aims to prove the following identity for every finite barrier raster B, read with 8-connected barriers and a 4-connected complement:

$$\#\{\text{enclosed 4-components of } \mathbb{Z}^2 \setminus B\} \;=\; C_8(B) - \chi(KB).$$

The plan is a three-rung ladder. L1 (this paper) is a local formula for $\chi(KB)$. L2 shows that adding one pixel preserves enclosed $- C_8 + \chi$. L3 obtains the bridge by induction on $|B|$. L1 is what makes the change in χ under a single insertion computable from the four windows around the new pixel.

2 Background and related work

Gray [2] introduced *bit-quads*: the 2×2 windows of a binary image. He showed that area, perimeter and the Euler number are "locally countable" from bit-quad statistics. For 8-connected foreground the Euler number is $(Q_1 - Q_3 - 2Q_D)/4$. Reading the foreground as 4-connected flips the sign of the diagonal term: two pixels that touch only at a corner then form two components, and $(6 - 0 + 2)/4 = 2$. In the 8-connected reading the two pixels meet at a corner of the closed squares, which is exactly the cubical complex KB used here. The (8,4) pairing of foreground and background connectivity is standard in digital topology, where it is the pairing under which a digital Jordan curve property holds; see the survey of Kong and Rosenfeld [4].

The formal development uses Lean 4 [1] and Mathlib [5]. We know of no prior machine-checked proof of the bit-quad formula, but we have not searched exhaustively, and we make no claim of novelty.

3 The sealed statement

A pixel $(i, j) \in \mathbb{Z}^2$ denotes the closed square $[i, i + 1] \times [j, j + 1]$. For a finite set B of pixels the sealed module `ViridisStatements.P1.BitQuad` defines:

- `vertices B`: the union of the four corners of every pixel;
- `hEdges B`: horizontal unit edges, named by their left end. Pixel (i, j) contributes (i, j) and $(i, j + 1)$;
- `vEdges B`: vertical unit edges, named by their lower end. Pixel (i, j) contributes (i, j) and $(i + 1, j)$;
- $\chi = |V| - (|E_h| + |E_v|) + |B|$, computed in $\mathbb{Z}$;
- the window at lattice point $v = (x, y)$, the bits $(a, b, c, d) = ([(x{-}1, y{-}1) \in B], [(x, y{-}1) \in B], [(x{-}1, y) \in B], [(x, y) \in B])$;
- Q_1, Q_3, Q_D: the number of $v \in$ `vertices B` whose window has one pixel, three pixels, or is a diagonal pair.

Every window that meets B is centred on a vertex, so counting over `vertices B` counts all non-empty windows.

Theorem 1 (L1, `P1.BitQuad`)**.** *For every finite $B \subset \mathbb{Z}^2$, $4\,\chi(KB) = Q_1(B) - Q_3(B) - 2\,Q_D(B)$.*

4 Proof architecture

4.1 Double counting by shifts

The only counting tool we need is the following lemma.

Lemma 2 (`card_filter_shift`)**.** *Let V, S be finite subsets of $\mathbb{Z}^2$, and let i, j be mutually inverse maps of $\mathbb{Z}^2$ with $j(S) \subseteq V$. Then $|\{v \in V : i(v) \in S\}| = |S|$.*

Indicator sums are filter cardinalities (Mathlib's `Finset.sum_boole`), so with $V =$ `vertices B` the lemma gives:

$$
\begin{aligned}
4|B| &= \sum_{v \in V} \big([a] + [b] + [c] + [d]\big) && \text{(each pixel is a window entry at each of its 4 corners),}\\
2|E_h| &= \sum_{v \in V} \big([v \in E_h] + [(x{-}1, y) \in E_h]\big) && \text{(right and left arms),}\\
2|E_v| &= \sum_{v \in V} \big([v \in E_v] + [(x, y{-}1) \in E_v]\big) && \text{(up and down arms),}\\
|V| &= \sum_{v \in V} 1. &&
\end{aligned}
$$

Each line uses four (or two) shift bijections. The side condition $j(S) \subseteq V$ says that every pixel's corners, and both endpoints of every edge, are vertices. Each of these is a small membership lemma.

4.2 Arms in terms of window bits

Unfolding the edge sets gives $e \in E_h \iff e \in B \vee (e_1, e_2 - 1) \in B$ and $e \in E_v \iff e \in B \vee (e_1 - 1, e_2) \in B$. At v the four arms are therefore $d \vee b$ (right), $c \vee a$ (left), $d \vee c$ (up) and $b \vee a$ (down).

4.3 The local identity

Summing the displays, $4\chi = \sum_{v \in V} \big(4 - 2\,\mathrm{arms}(v) + n(v)\big)$ with $n = [a] + [b] + [c] + [d]$. So it suffices to show, for each vertex,

$$4 - 2\,\mathrm{arms}(v) + n(v) = [n = 1] - [n = 3] - 2[\mathrm{diag}].$$

Table 1 lists the pattern classes. The identity *fails* on the empty window ($4 \neq 0$). We therefore use a lemma (`window_nonempty`): each vertex is a corner of some pixel of B, so its window is non-empty. In Lean we split on the four membership propositions (16 cases) and close each case with `simp_all`. The brief suggested `decide` on Booleans. Our first draft took that route, and its `decide` correctly reported the unrestricted identity false. That is how we found the empty-window case.

Pattern class	n	arms	$4 - 2\,\mathrm{arms} + n$	$[n{=}1] - [n{=}3] - 2[\mathrm{diag}]$
empty (never at a vertex)	0	0	4	0
one pixel	1	2	1	1
adjacent pair	2	3	0	0
diagonal pair	2	4	-2	-2
three pixels	3	4	-1	-1
full	4	4	0	0

Table 1: Local summand by window pattern class.

5 Results: the Lean development

The file `proofs/P1.BitQuad.lean` proves `Candidate.main : ViridisStatements.P1.BitQuad.Statement`. It has 159 lines and imports only Mathlib and the sealed module. The project's local checker

(denylist, compile, statement match, axioms, kernel replay) reported that it passed, with axioms `propext`, `Classical.choice` and `Quot.sound`. The authoritative judgement is the verifier's certification page. No Aristotle job was needed. The key lemmas follow.

```
theorem card_filter_shift (V S : Finset (ℤ ×ℤ)) (i j : ℤ×ℤ→ℤ×ℤ)
    (hij : ∀p, i (j p) = p) (hji : ∀v, j (i v) = v)
    (hS : ∀p ∈S, j p ∈V) :
    (V.filter fun v => i v ∈S).card = S.card := by
  apply Finset.card_nbij' i j
  ·intro v hv
    simp only [Finset.coe_filter, Set.mem_setOf_eq] at hv
    exact hv.2
  ·intro p hp
    simp only [Finset.coe_filter, Set.mem_setOf_eq]
    exact ⟨hS p hp, by rw [hij]; exact hp⟩
  ·intro v _; exact hji v
  ·intro p _; exact hij p

theorem mem_hEdges (B : Finset Pixel) (e : ℤ×ℤ) :
    e ∈hEdges B ↔e ∈B ∨(e.1, e.2 - 1) ∈B

theorem window_nonempty (B : Finset Pixel) (v : ℤ×ℤ) (hv : v ∈vertices B) :
    (v.1 - 1, v.2 - 1) ∈B ∨(v.1, v.2 - 1) ∈B ∨(v.1 - 1, v.2) ∈B ∨(v.1, v.2) ∈B
```

The end of the main proof turns the global identity into the pointwise one:

```
  rw [key, hQ1, hQ3, hQD, Finset.mul_sum, ←Finset.sum_sub_distrib,
      ←Finset.sum_sub_distrib]
  apply Finset.sum_congr rfl
  intro v hv
  have hne := window_nonempty B v hv
  simp only [mem_hEdges, mem_vEdges]
  unfold windowCount isDiagonal bits
  by_cases h1 : (v.1 - 1, v.2 - 1) ∈B <;> by_cases h2 : (v.1, v.2 - 1) ∈B <;>
    by_cases h3 : (v.1 - 1, v.2) ∈B <;> by_cases h4 : (v.1, v.2) ∈B <;>
    simp_all
```

Here `key` rewrites $4\,\chi$ as the sum of the local summand over `vertices B`. It is obtained by distributing the sum and substituting the eight shift identities. The proof concludes with `ring`.

6 How L1 serves the bridge

L2 asks that inserting a pixel $p \notin B$ preserve enclosed $- C_8 + \chi$. By Theorem 1, $\Delta\chi$ is determined by the four windows centred on the corners of p. Only those windows change, so $4\,\Delta\chi = \Delta Q_1 - \Delta Q_3 - 2\,\Delta Q_D$ is a finite local computation over 2^8 neighbourhood patterns of p. The non-local part of L2 is the change in C_8 and in the number of enclosed components. Two 8-components merging through p can close off a new region. That part is not touched here. We expect that the natural Lean statement of L2 will be phrased in terms of the local $\Delta\chi$ that L1 supplies. The three controls listed in the project README (the 3×3 ring witness, the 2×2 block and the four-arm plus) give concrete checks: the sealed module verifies $\chi = 1$ for the 2×2 block and $\chi = 0$ for the ring by computation.

7 Limitations

- L1 is a statement about the cubical complex counted combinatorially. It says nothing about connected components or enclosure. The topological content of the bridge sits entirely in L2.
- The statement fixes the 8-connected reading of the foreground. The 4-connected variant ($+2Q_D$) is not formalised here.
- L1 by itself does not discharge the `FaceCount` hypothesis of *Trees Cannot Cut*. It also does not address the raster-to-graph correspondence, whose gap the 2×2 block control exposes (circuit rank 3, nothing enclosed).

8 Reproducibility

Toolchain `leanprover/lean4:v4.28.0`; Mathlib commit `8f9d9cff6bd728b17a24e163c9402775d9e6a365`.
The sealed statement is `P1.BitQuad.lean` (SHA-256 `cd3cdeba48f67f9545fb6945c87037893aa98b018922d3d0b72`
To reproduce, run `lean proofs/P1.BitQuad.lean` with `LEAN_PATH` including Mathlib and the sealed modules. Compilation takes about 10 s on the session machine. The proof contains no `sorry`, `native_decide` or custom axioms.

9 Changelog

- 2026-10-27 (P1-T001): first version. Lean proof of L1 (`P1.BitQuad`); proof architecture; role of L1 in the L2/L3 ladder.

References

[1] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37.

[2] Stephen B. Gray. Local properties of binary images in two dimensions. *IEEE Transactions on Computers*, C-20(5):551–561, 1971. doi: 10.1109/T-C.1971.223289.

[3] Justin D. Hart. Trees cannot cut: Circuit rank as the design variable reconciling habitat connectivity and fuel discontinuity in managed forest landscapes. Zenodo, Viridis LLC, 2026.

[4] T. Yung Kong and Azriel Rosenfeld. Digital topology: Introduction and survey. *Computer Vision, Graphics, and Image Processing*, 48(3):357–393, 1989. doi: 10.1016/0734-189X(89)90147-3.

[5] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020. doi: 10.1145/3372885.3373824.